Projektdokumentation

N. Schönmeyer H. Staab N. Henzen

J. Stripp C. Henz

Technische Hochschule Aschaffenburg

Fakultät IWIN '25/26
Schwerpunkt Data Science
Projekt 2025/2026

Februar 2026

Inhaltsverzeichnis

1 Einleitung **4**

1.1 Motivation und Hintergrund ... 4

1.2 Ziel des Projekts ... 4

1.3 Projektumfang ... 5

1.4 Komponenten des Reinforcement Learning ... 5

2 Projektaufbau **8**

2.1 Systemübersicht ... 8

2.2 Architektur ... 8

3 Hardware **10**

3.1 Arduino ... 10

3.2 Kamera ... 11

3.3 Sonstige Hardware ... 13

4 Kameratracking **14**

4.1 Übersicht der Aufgaben ... 14

4.2 Hardware-Initialisierung ... 14

4.3 Bildverarbeitung ... 16

4.4 Crash-Detection ... 18

4.5 Konfiguration ... 20

4.6 Integration & Datenfluss ... 21

4.7 Auto-Reset-System ... 22

5 Simulator **23**

5.1 Simulationsumgebung ... 23

5.2 Simulationsablauf ... 24

5.3 Modellierung der Carrera Strecke ... 25

5.4 Annäherung des Physikalischen Fahrzeugmodells ... 26

5.5 Machine Learning Framework ... 28

5.6 Rolle des Simulators im Gesamtsystem ... 30

6 Reinforcement Learning Prozess **32**

6.1 Aktions- & Zustandsraum ... 32

6.2 Reward-Funktion ... 33

6.3 Trainingsablauf ... 35

7 Herausforderungen **37**

7.1 Lichtverhältnisse ... 37

7.2 Kameraqualität ... 37

8 Evaluation **38**
8.1 Trainingsmetriken ... 38
8.2 Qualitative Bewertung ... 39
8.3 Sim-to-Real Transfer ... 39
9 Projektorganisation **40**
9.1 Teamstruktur und Arbeitsmethodik ... 40
10 Fazit und Ausblick **41**
10.1 Fazit ... 41
10.2 Limitationen ... 41
10.3 Ausblick ... 42
11 Begriffserklärungen **44**
12 Aufbauanleitung **46**
12.1 Voraussetzungen ... 46
12.2 Installation ... 46
12.3 Schnellstart ... 47
12.4 Fehlerbehebung ... 47
13 Literatur **49**

1 Einleitung

1.1 Motivation und Hintergrund

Autonome Systeme gewinnen in immer mehr Bereichen an Bedeutung – von selbstfahrenden Autos über Industrieroboter bis hin zu Drohnen. Ein zentrales Problem bei der Entwicklung solcher Systeme ist das Training: Wie bringt man einer Maschine bei, komplexe Entscheidungen in Echtzeit zu treffen? Eine vielversprechende Antwort liefert das Reinforcement Learning (RL), bei dem ein lernender Agent durch Versuch und Irrtum eine optimale Strategie entwickelt.

Dieses Projekt wendet Reinforcement Learning auf ein zugängliches, aber dennoch anspruchsvolles Problem an: die autonome Steuerung eines Carrera-Fahrzeugs auf einer Slot-Car-Rennbahn. Was auf den ersten Blick wie ein Spielzeug wirkt, stellt tatsächlich eine Reihe interessanter Herausforderungen dar: Das Fahrzeug muss präzise gesteuert werden, um nicht aus der Kurve zu fliegen, gleichzeitig soll es möglichst schnell sein. Diese Balance zwischen Geschwindigkeit und Stabilität ist ein klassisches Optimierungsproblem, das sich hervorragend für RL eignet.

1.2 Ziel des Projekts

Ziel dieses Projekts ist die Anwendung von Reinforcement Learning zur autonomen Steuerung eines Carrera-Fahrzeugs mit dem Ziel, eine vorgegebene Rennstrecke möglichst effizient zu befahren. Dabei soll ein lernender Agent in der Lage sein, durch Interaktion mit seiner Umgebung eine Fahrstrategie zu entwickeln, die eine minimale Rundenzeit erreicht, ohne dass das Fahrzeug die Strecke verlässt.

Um den hohen Trainingsaufwand und die Risiken eines direkten Trainings auf der realen Strecke zu vermeiden, wird der Lernprozess überwiegend in einer realitätsnahen Simulationsumgebung durchgeführt. Dieser Ansatz, oft als „Sim-to-Real Transfer“ bezeichnet, ermöglicht es, tausende von Trainingsrunden in wenigen Minuten durchzuführen, bevor das Gelernte auf die physische Strecke übertragen wird.

1.3 Projektumfang

Das Projekt umfasst mehrere Teilbereiche, die zusammen ein vollständiges System bilden:

- **Kamerabasiertes Tracking**: Ein Python-System erkennt die Position des Fahrzeugs auf der Strecke in Echtzeit mittels Farbanalyse und April-Tag-Markierungen.
- **3D-Simulation**: Eine in Godot entwickelte Simulationsumgebung bildet die physikalischen Eigenschaften der Carrera-Bahn nach und ermöglicht schnelles, risikofreies Training.
- **Hardware-Steuerung**: Ein Arduino-basiertes System wandelt die Entscheidungen des RL-Agenten in elektrische Signale um, die das reale Fahrzeug steuern.
- **Reinforcement Learning**: Der PPO-Algorithmus (Proximal Policy Optimization) lernt eine Fahrstrategie, die Geschwindigkeit maximiert und Abstürze vermeidet.

1.4 Komponenten des Reinforcement Learning

1.4.1 Agent

Der Agent stellt die lernende Instanz des Systems dar. In unserem Projekt übernimmt der Agent die Aufgabe, das Carrera-Fahrzeug autonom zu steuern. Basierend auf den vom Simulator bereitgestellten Zustandsinformationen trifft der Agent Entscheidungen über die anzuwendende Steueraktion. Ziel des Agenten ist es, eine Strategie zu erlernen, die langfristig eine maximale kumulative Belohnung erzielt, was in unserem Fall einer möglichst schnellen und stabilen Fahrt entspricht.

1.4.2 Umwelt

Die Umwelt umfasst alle äußeren Gegebenheiten, mit denen der Agent interagiert. In diesem Projekt wird die Umwelt durch die in Godot implementierte Simulationsumgebung, sowie die physische Rennstrecke repräsentiert. Die Umwelt reagiert auf die vom Agenten gewählten Aktionen, aktualisiert den Systemzustand und liefert entsprechendes Feedback in Form von Belohnungen.

1.4.3 Zustand

Der Zustand beschreibt die aktuelle Situation der Umwelt aus Sicht des Agenten. Er enthält alle relevanten Informationen, die für eine Entscheidungsfindung notwendig sind. In unserem Projekt umfasst der Zustandsraum unter anderem Informationen zur aktuellen Geschwindigkeit des Fahrzeugs, zur Position auf der Strecke sowie zu streckenbezogenen Merkmalen wie der Annäherung an Kurven. Ziel der Zustandsdefinition ist es, dem Agenten eine möglichst vollständige, aber zugleich kompakte Repräsentation der aktuellen Fahrsituation bereitzustellen.

1.4.4 Aktion

Eine Aktion beschreibt eine vom Agenten ausführbare Steuerungsentscheidung. In unserem Fall besteht der Aktionsraum aus einem **kontinuierlichen Throttle-Wert** zwischen 0 und 1, der die Beschleunigung des Fahrzeugs bestimmt. Die Entscheidung für einen kontinuierlichen statt diskreten Aktionsraum wurde getroffen, nachdem sich zeigte, dass das Modell bei diskreten Stufen (z.B. 0, 25, 50, 75, 100) dazu neigte, sich auf sichere mittlere Werte „einzupendeln“ und keine optimale Fahrstrategie entwickelte. Durch die Wahl einer Aktion beeinflusst der Agent direkt das Fahrverhalten und damit den weiteren Verlauf der Episode.

1.4.5 Reward

Der Reward stellt das zentrale Feedbacksignal im Reinforcement Learning dar und bewertet die Qualität einer ausgeführten Aktion im jeweiligen Zustand. In unserem Projekt ist die Reward-Funktion so gestaltet, dass sie den Agenten zu einer möglichst kurzen Rundenzeit motiviert, während das Verlassen der Strecke oder instabiles Fahrverhalten negativ bewertet wird.

1.4.6 gewählter RL-Algorithmus

Für das Training des Agenten wurde Proximal Policy Optimization (PPO) gewählt. Der Hauptgrund für diese Entscheidung liegt in der hohen Stabilität des Algorithmus bei kontinuierlichen und rauschbehafteten Steuerungsaufgaben. PPO gehört zu den policy-basierten Reinforcement-Learning-Verfahren und optimiert die Entscheidungsstrategie direkt, anstatt ausschließlich Zustandswerte zu approximieren. Dies ist insbesondere für unser Szenario von Vorteil, da wir zuerst den Zustandsraum nicht durch eine Diskretion abstrahieren wollten.

Zusätzlich zeigte sich PPO als gut geeignet für die Kombination aus Simulations- und Realumgebung, da der Algorithmus vergleichsweise robust gegenüber Ungenauigkeiten im Zustandsraum ist. Im Vergleich zu wertbasierten Verfahren ermöglichte PPO einen zuverlässigeren Lernfortschritt und eine bessere Konvergenz, ohne eine aufwändige manuelle Feinabstimmung der Lernparameter zu erfordern.

Insgesamt stellte PPO einen guten Kompromiss zwischen Trainingsstabilität und Leistungsfähigkeit dar und eignete sich daher besonders für die Anforderungen unseres Projekts.

Eine interne Diskussion herrschte allerdings darüber, ob ein Umstieg auf ein Q-Learning Modell sinnvoll sein könnte. Hierfür könnte man den Zustandsraum diskretisieren, was durch Streckenteile recht intuitiv ist. Auf diesen diskreten Zustands- und Aktionsräumen kann Q-Learning dann effizient trainiert werden.

Allerdings wurde das nicht umgesetzt, da unklar war, ob Q-Learning stabil genug ist, um in unserem Fall zu funktionieren, und für ausreichendes Testen die Zeit zu gering war.

2 Projektaufbau

2.1 Systemübersicht

Bevor wir in die technischen Details einsteigen, ist es wichtig, das Gesamtbild zu verstehen. Das System besteht aus drei Hauptkomponenten, die zusammenarbeiten, um das Fahrzeug autonom zu steuern. Jede Komponente hat eine klar definierte Aufgabe und kommuniziert über standardisierte Schnittstellen mit den anderen.

Stellen Sie sich das System wie folgt vor: Eine Kamera beobachtet die Rennstrecke von oben und erkennt, wo sich das Fahrzeug befindet. Diese Information wird an einen Computer gesendet, der entweder eine Simulation der Strecke laufen lässt oder direkt das reale Fahrzeug steuert. Der RL-Agent, das „Gehirn" des Systems, analysiert die aktuelle Situation und entscheidet, wie stark beschleunigt werden soll. Diese Entscheidung wird dann über einen Arduino in ein elektrisches Signal umgewandelt, das das Fahrzeug antreibt.

2.2 Architektur

Das System besteht aus drei Hauptkomponenten, die über Netzwerkprotokolle miteinander kommunizieren:

2.2.1 Python-Tracking-System (src/)

Das Python-System ist das „Auge" des Gesamtsystems. Es erfasst über eine Webcam das Geschehen auf der Strecke und verarbeitet die Bilddaten in Echtzeit. Die Hauptaufgaben sind die Erkennung der Fahrzeugposition, die Identifikation der Streckenteile mittels AprilTags und die Erkennung von Unfällen (Crashes).

Die wichtigsten Module sind:

- `camera/`: Verantwortlich für die Kamera-Initialisierung und die Grundlagen der Bildverarbeitung. Hier wird sichergestellt, dass die Kamera mit optimalen Einstellungen läuft und minimale Latenz liefert.
- `tracking/`: Enthält die Kernlogik für AprilTag-Erkennung, Streckenmapping, Crash-Detection und das automatische Reset-System. Diese Module arbeiten zusammen, um ein vollständiges Bild der aktuellen Situation zu erstellen.
- `network/`: Kümmert sich um die TCP-Kommunikation mit dem Godot-Simulator sowie die serielle Kommunikation mit dem Arduino.

- `config.py`: Eine zentrale Konfigurationsdatei, in der alle wichtigen Parameter gesammelt sind. Dies ermöglicht schnelle Anpassungen ohne Änderung des Quellcodes.

2.2.2 Godot-Simulator (CarreraSimulator/)

Der Simulator ist das Herzstück des Trainingsprozesses. Er stellt eine virtuelle Nachbildung der Carrera-Bahn dar, in der der RL-Agent gefahrlos und mit hoher Geschwindigkeit trainieren kann. In der Simulation können tausende Runden in wenigen Minuten gefahren werden – etwas, das auf der realen Strecke Stunden dauern würde.

Die wichtigsten Bestandteile sind:

- `Tracks/track_builder_2/`: Hier befinden sich die aktuellen Strecken- und Fahrzeugskripte. Das Fahrzeugmodell simuliert physikalisches Verhalten wie Beschleunigung, Bremsen und das „Abfliegen“ in Kurven.
- `Singletons/network_manager.gd`: Ein globaler Manager, der die Netzwerkkommunikation mit dem Python-System koordiniert.
- `Trackloader/`: Ermöglicht das dynamische Laden von Strecken basierend auf den vom Python-System erkannten AprilTags.
- `addons/godot_rl_agents/`: Ein externes Plugin, das die Integration von Reinforcement Learning in Godot ermöglicht.

2.2.3 Kommunikationsfluss

Die drei Komponenten kommunizieren über definierte Protokolle:

1. **Python → Godot**: Das Python-System sendet Informationen über den aktuellen Zustand an den Simulator:
    - `track_order`: Die Reihenfolge der erkannten Streckenteile beim Start
    - `progress`: Der aktuelle Fortschritt des Fahrzeugs auf der Strecke (0-100%)
    - `crash`: Ein Signal, wenn ein Unfall erkannt wurde
2. **Godot → Python**: Der Simulator (bzw. der RL-Agent) sendet Steuerungsbefehle zurück:
    - `signal_rate_level`: Der gewünschte Throttle-Wert (0-100), der die Geschwindigkeit bestimmt

3. **Python → Arduino**: Die Throttle-Werte werden über eine serielle Verbindung (USB) an den Arduino gesendet, der sie in PWM-Signale für das Fahrzeug umwandelt.

3 Hardware

In diesem Kapitel werden die physischen Komponenten des Systems beschrieben. Die Hardware bildet die Brücke zwischen der digitalen Welt des RL-Agenten und der realen Carrera-Bahn.

3.1 Arduino

3.1.1 Rolle im System

Der Arduino Uno dient als Schnittstelle zwischen dem RL-System und der Carrera GO Bahn. Er fungiert als „Übersetzer", der die digitalen Befehle des Computers (Geschwindigkeitswerte zwischen 0 und 100) in elektrische Signale umwandelt, die das Fahrzeug antreiben können.

Warum ein Arduino? Die Carrera GO Bahn wird normalerweise über einen Handregler gesteuert, der einen variablen Widerstand enthält. Je stärker man drückt, desto mehr Strom fließt zum Fahrzeug. Um diese manuelle Steuerung zu automatisieren, muss ein elektrisches Signal erzeugt werden, das den Handregler ersetzt. Der Arduino ist ideal für diese Aufgabe, da er PWM-Signale (Pulsweitenmodulation) erzeugen kann, die eine präzise Geschwindigkeitssteuerung ermöglichen.

3.1.2 Technische Details

Der Arduino empfängt Geschwindigkeitswerte (0-100) über die serielle Schnittstelle (USB-Verbindung zum Computer) mit einer Baudrate von 9600. Diese Werte werden dann in PWM-Signale umgewandelt:

- Eingehende Werte werden von 0-100 auf PWM-Werte 0-255 skaliert. Dies geschieht mit der eingebauten `map()`-Funktion, die eine lineare Umrechnung durchführt.
- PWM-Ausgabe erfolgt über Pin 11 (`CARRERA_PIN`). Dieser Pin wurde gewählt, da er PWM-fähig ist und eine ausreichend hohe Frequenz für eine sanfte Motorsteuerung bietet.

- Zusätzliche Status-LEDs auf Pin 2 (rot) und Pin 4 (blau) zeigen den aktuellen Geschwindigkeitsbereich visuell an. Dies hilft beim Debugging und gibt dem Benutzer Feedback über den Systemzustand.
- Ein STATUS-Kommando ermöglicht die Abfrage des aktuellen Zustands, was für Diagnose und Debugging nützlich ist.

3.1.3 Kommunikationsprotokoll

Die Kommunikation mit dem Arduino folgt einem einfachen textbasierten Protokoll:

- **Geschwindigkeitsbefehl**: Eine Zahl zwischen 0 und 100 als ASCII-String, gefolgt von einem Newline-Zeichen (z.B. `75\n`)
- **STATUS-Abfrage**: Der Befehl `STATUS\n` gibt eine detaillierte Antwort zurück: `STATUS:speed=X,errors=Y,uptime=Z,pin=11`
- **Fehlerbehandlung**: Ungültige Werte (außerhalb 0-100) werden automatisch auf den gültigen Bereich korrigiert, um Fehlfunktionen zu vermeiden

3.2 Kamera

3.2.1 Bedeutung für das System

Die Kamera ist das „Auge“ des Systems. Sie ermöglicht es, die Position des Fahrzeugs auf der Strecke zu erkennen, ohne am Fahrzeug selbst Sensoren anbringen zu müssen. Dies ist wichtig, da zusätzliches Gewicht oder Elektronik am kleinen Carrera-Fahrzeug dessen Fahrverhalten negativ beeinflussen würde.

3.2.2 Gewähltes Modell

Für das Tracking wird eine hochwertige Webcam mit stabiler Bildqualität und guter Auflösung benötigt. Im Projekt wird eine **Logitech Brio 4K** verwendet, da diese die erforderliche Bildschärfe für zuverlässige AprilTag-Erkennung liefert.

3.2.3 Kamera-Upgrade: Lessons Learned

Ein wichtiger Erfahrungswert aus dem Projekt ist die Bedeutung der Kamera-Spezifikationen für das Tracking. Die ursprünglich verwendete Kamera konnte lediglich **1080p bei 30 Frames pro Sekunde** liefern, was sich als kritische Limitation erwies.

Das Problem manifestierte sich folgendermaßen: Bei einer Bildrate von nur 30 FPS und einem schnell fahrenden Carrera-Fahrzeug bewegt sich das Fahrzeug zwischen zwei aufeinanderfolgenden Frames eine erhebliche Strecke. Dies führte zu mehreren Schwierigkeiten:

1. **Tracking-Verlust**: Die Farberkennung des grünen Markers funktionierte unzuverlässig, da das Fahrzeug zwischen Frames „springen“ konnte und die Bewegungsvorhersage fehlschlug.
2. **Ungenaue Geschwindigkeitsberechnung**: Die Crash-Detection basiert auf der Berechnung der Fahrzeuggeschwindigkeit aus Positionsänderungen. Bei wenigen Frames pro Sekunde wird diese Berechnung ungenau und anfällig für Fehlalarme.
3. **Erhöhte Latenz**: Mit weniger verfügbaren Frames ist das System weniger reaktionsfähig, was bei einem schnellen Fahrzeug kritisch sein kann.

Die **Logitech Brio 4K** löst diese Probleme durch höhere Bildraten (bis zu 60 FPS bei 1080p) und bessere Bildqualität. Diese Erfahrung unterstreicht, dass bei Echtzeit-Tracking-Anwendungen die Bildrate oft wichtiger ist als die reine Auflösung.

3.2.4 Empfohlene Einstellungen

Die Kamera muss sorgfältig konfiguriert werden, um optimale Ergebnisse zu erzielen:

- **Auflösung**: FHD (1920x1080) bietet einen guten Kompromiss zwischen Detailgenauigkeit und Verarbeitungsgeschwindigkeit. Höhere Auflösungen würden zwar mehr Details liefern, aber auch mehr Rechenzeit benötigen.
- **Belichtung**: Eine kurze Belichtungszeit (`EXPOSURE = -6` unter Windows) ist essenziell. Dieser Wert wurde durch iteratives Testen ermittelt und stellt einen Mittelweg dar: Das Bild muss hell genug für zuverlässiges Tracking sein, gleichzeitig darf die Belichtungszeit nicht so lang sein, dass das fahrende Auto als unscharfer „Schweif“ erscheint. Bei zu langer Belichtung verschmiert die grüne Markierung auf dem Fahrzeug, was die Konturerkennung unmöglich macht.

- **Puffergröße**: Der Kamerapuffer wird auf 1 Frame begrenzt. Ohne diese Einstellung würde die Kamera mehrere Bilder zwischenspeichern, was zu einer Verzögerung zwischen Realität und erkanntem Bild führt.
- **Positionierung**: Durch das automatische Scanning und die anschließende Abbildung im Simulator ist der genaue Abstand der Kamera zur Strecke nicht kritisch, solange alle Streckenteile vollständig im Sichtfeld (Field of View) der Kamera liegen und die AprilTags in der korrekten Reihenfolge erkannt werden können. Dies bietet Flexibilität bei der Installation und macht das System robust gegenüber unterschiedlichen Aufbauten.

3.3 Sonstige Hardware

Neben Arduino und Kamera werden weitere Komponenten benötigt:

- **Carrera GO**: Die Slot-Car-Rennbahn bildet die physische Grundlage des Projekts. Die Carrera GO war für dieses Projekt vorgegeben und stellt ein analoges System dar. Im Gegensatz zu digitalen Carrera-Systemen (wie Carrera Digital 132/124), die über digitale Signale mehrere Fahrzeuge auf einer Spur steuern können, erfolgt die Geschwindigkeitsregelung bei Carrera GO rein analog über die Spannung. Dies macht die Integration mit dem Arduino einfacher, da lediglich ein PWM-Signal in ein analoges Spannungsniveau umgewandelt werden muss.
- **Softbox**: Gleichmäßige Beleuchtung ist entscheidend für die Bildverarbeitung. Eine Softbox eliminiert harte Schatten und Reflexionen, die die Farberkennung stören würden. Ohne gute Beleuchtung schwanken die Erkennungsraten erheblich. Besonders kritisch ist die Beleuchtung für die AprilTag-Erkennung, die bei extremen Lichtverhältnissen (zu hell oder zu dunkel) massive Probleme bereitet.
- **Markierung auf Fahrzeug**: Eine grüne Fläche auf dem Autodach dient als Tracking-Marker. Die Farbe Grün wurde von einer Vorgängergruppe gewählt und übernommen, da sie sich gut von den typischen Farben der Strecke (grau, schwarz) und des Hintergrunds (heller Boden ohne grüne Elemente) abhebt.
- **Breadboard**: Ermöglicht die flexible Verbindung zwischen Arduino und Carrera-Bahn ohne Löten. Dies erleichtert Anpassungen und Experimente.

4 Kameratracking

Das Kameratracking-System ist verantwortlich für die Erfassung und Verarbeitung der visuellen Informationen von der Rennstrecke. Es beantwortet die zentrale Frage: „Wo befindet sich das Fahrzeug gerade?“ Diese Information ist essentiell für den RL-Agenten, da er nur auf Basis des aktuellen Zustands sinnvolle Entscheidungen treffen kann.

4.1 Übersicht der Aufgaben

Das Tracking-System muss mehrere Aufgaben gleichzeitig bewältigen:

- **Low-Latency Image Capture**: Die Verzögerung zwischen dem realen Geschehen und der Bildverarbeitung muss minimal sein. Wenn das System zu langsam reagiert, ist das Fahrzeug bereits an einer anderen Position als erkannt, was zu fehlerhaften Steuerungsentscheidungen führt.
- **Objekterkennung**: Das Fahrzeug muss in jedem Kamerabild zuverlässig identifiziert werden. Dies geschieht über die grüne Markierung auf dem Fahrzeugdach mittels Farbraum-Analyse.
- **Streckenerkennung**: Die AprilTags auf den einzelnen Streckenteilen müssen erkannt und in die richtige Reihenfolge gebracht werden. Zusätzlich muss die Fahrtrichtung (im Uhrzeigersinn oder gegen den Uhrzeigersinn) ermittelt werden.

4.2 Hardware-Initialisierung

Die Kamera-Initialisierung ist komplexer, als es zunächst scheint. Verschiedene Betriebssysteme haben unterschiedliche Schnittstellen für den Kamerazugriff, und die Wahl des richtigen Backends kann erhebliche Auswirkungen auf Performance und Zuverlässigkeit haben.

Die Hardware-Initialisierung geschieht in der **camera_manager.py**. Die Verbindung zur Kamera geschieht dynamisch, abhängig vom Betriebssystem des Computers, um bestmögliche Performance und Kompatibilität zu gewährleisten.

4.2.1 Plattform-Spezifische Backends

Jedes Betriebssystem bietet unterschiedliche Möglichkeiten, auf Kamerabilder zuzugreifen:

4.2.1.1 Windows

Windows bietet die DirectShow-API für den direkten Hardware-Zugriff. Dies ermöglicht:

- Optimierung für hohe Geschwindigkeit durch direkten Zugriff auf die Kamera-Hardware
- Spezifische Belichtungsparameter: Die Exposure wird auf „−6“ gesetzt (kurze Belichtungszeit) und die Brightness auf „1000“ für optimale Helligkeit
- Diese Einstellungen erzwingen eine kurze Belichtungszeit, um die Bewegungsunschärfe bei hohen Geschwindigkeiten zu eliminieren

4.2.1.2 MacOS

Auf Apple-Systemen wird das native AVFoundation Framework verwendet, das tiefe Integration mit der Hardware bietet.

4.2.1.3 Linux

Linux-Systeme nutzen das Standard-Backend (V4L2), das auf den meisten Distributionen verfügbar ist.

4.2.2 Latenz-Management

Ein kritischer Aspekt der Bildverarbeitung ist die Latenz – die Zeit zwischen dem realen Ereignis und seiner Verarbeitung. Der interne Puffer der Kamera wird betriebssystemunabhängig auf ein Frame begrenzt.

Warum ist das wichtig? Ohne diese Einstellung würde die Kamera standardmäßig mehrere Frames zwischenspeichern (typischerweise 3-5). Das bedeutet, dass das verarbeitete Bild nicht das aktuellste ist, sondern eines aus der Vergangenheit. Bei einem Fahrzeug, das sich schnell bewegt, kann das den Unterschied zwischen erfolgreicher Steuerung und einem Crash ausmachen.

4.2.3 Fehlerbehandlung

Kameras sind nicht immer zuverlässig – USB-Verbindungen können instabil sein, oder die Kamera ist beim Programmstart noch nicht bereit. Während des Initialisierungsprozesses wird durch eine Retry-Logik bis zu dreimal versucht, einen validen Frame zu lesen. Erst nach dem dritten fehlgeschlagenen Versuch wird ein Fehler gemeldet und das System beendet.

4.3 Bildverarbeitung

Die eigentliche Bildverarbeitung ist die Kernaufgabe des Tracking-Systems. Sie findet in der Datei **color_detector.py** statt und wird von der Klasse **CarTracker** durchgeführt.

4.3.1 Warum HSV statt RGB?

Die Bildverarbeitung nutzt den HSV-Farbraum (Hue, Saturation, Value) anstelle des bekannteren RGB-Farbraums (Red, Green, Blue). Der Grund: HSV ist robuster gegenüber Helligkeitsschwankungen.

Im RGB-Farbraum ändert sich die Farbe eines Objekts deutlich, wenn das Licht heller oder dunkler wird. Im HSV-Farbraum hingegen bleibt der Farbton (Hue) weitgehend konstant – nur die Helligkeit (Value) ändert sich. Das macht die Erkennung der grünen Fahrzeugmarkierung stabiler, selbst wenn die Beleuchtung nicht perfekt gleichmäßig ist.

4.3.2 Fahrzeug Tracking

Die Methode **find_car()** ist für die Erkennung des Fahrzeugs zuständig:

4.3.2.1 Wahl der Tracking-Farbe

Die Farbe Grün wurde als Tracking-Marker gewählt, da sie sich optimal von den typischen Farben der Rennstrecke (grau, schwarz) und des Hintergrunds (heller Boden) abhebt. Diese Entscheidung wurde von einer Vorgängergruppe getroffen, die das Fahrzeug bereits mit einer grünen Dachmarkierung versehen hatte.

Während der Entwicklung wurden alternative Farben für andere Tracking-Aufgaben (z.B. Streckenerkennung) evaluiert:

- **Blau** erwies sich als problematisch, da die Farbe sehr anfällig für Artefakte in Schattenbereichen war. Dunkle Schatten wurden häufig fälschlicherweise als Blau interpretiert.
- **Gelb** zeigte bessere Ergebnisse als Blau, wurde jedoch in hellem Licht oft fehlerhaft erkannt. Aufgrund der relativen Zuverlässigkeit wird Gelb für einen Orientierungspunkt am Start verwendet, der die Fahrtrichtung markiert.

Um sicherzustellen, dass die Grün-Erkennung nicht mit Umgebungselementen interferiert, wurde ein dedizierter Kalibrierungsprozess implementiert. Mithilfe

von OpenCV werden verschiedene Masken und HSV-Schwellwerte bestimmt, die eine zuverlässige Unterscheidung zwischen Fahrzeugmarkierung und Hintergrund ermöglichen.

4.3.2.2 Erkennung des Fahrzeugs

Der Erkennungsprozess läuft in mehreren Schritten ab:

1. **Farbkonvertierung**: Das Kamerabild wird von RGB in den HSV-Farbraum konvertiert.
2. **Farbfilterung**: Durch einen binären Filter werden alle Pixel isoliert, die im definierten grünen Farbbereich liegen. Das Ergebnis ist ein Schwarz-Weiß-Bild, in dem nur die grünen Bereiche weiß erscheinen.
3. **Rauschunterdrückung**: Das gefilterte Bild enthält oft kleine Störungen – einzelne Pixel, die fälschlicherweise als grün erkannt wurden, oder kleine Lücken in der eigentlich zusammenhängenden Fläche. Diese werden durch morphologische Operationen korrigiert:
   - **MORPH_OPEN** entfernt kleine Rauschpixel (einzelne weiße Punkte werden zu schwarz)
   - **MORPH_DILATE** schließt verbleibende Lücken im Fahrzeugkörper (kleine schwarze Bereiche in weißen Flächen werden gefüllt)
4. **Konturerkennung**: Im bereinigten Bild werden zusammenhängende weiße Bereiche (Konturen) gesucht. Die größte gefundene Kontur wird als das Fahrzeug interpretiert.
5. **Zentrumsberechnung**: Aus der größten Kontur wird mittels Bildmomenten das exakte Zentrum des Autos berechnet. Bildmomente sind mathematische Eigenschaften einer Form, die unter anderem ihren Schwerpunkt beschreiben.

4.3.3 Referenzpunkte der Strecke

Neben dem Fahrzeug werden auch Referenzpunkte auf der Strecke erkannt. Die Methode `find_yellow_dot()` erkennt gelbe Markierungen, die als statische Wegpunkte dienen. Diese Punkte helfen bei der Kalibrierung der Streckengeometrie und bei der Bestimmung der Streckenreihenfolge beim Systemstart. Die Wahl von Gelb für diesen Zweck basiert auf den Evaluierungsergebnissen: Obwohl Gelb bei hellem Licht Probleme verursachen kann, ist es für einen

statischen Punkt, der nur beim Systemstart erkannt werden muss, ausreichend zuverlässig.

4.3.4 AprilTag-Erkennung

AprilTags sind spezielle 2D-Barcodes, die für robotische Anwendungen entwickelt wurden. Sie ermöglichen eine präzise Identifikation und Lokalisierung von Markern in Kamerabildern. Jeder Streckentyp (Gerade, Linkskurve, Rechtskurve, etc.) ist mit einem spezifischen AprilTag markiert.

4.3.4.1 Wahl der Tag-Familie

Die Erkennung erfolgt mittels der `pupil_apriltags`-Bibliothek mit der Tag-Familie `tag16h5`. Diese Familie wurde ursprünglich gewählt, da die Annahme bestand, dass eine kleinere Tag-Familie (weniger mögliche Tags) zu einer besseren Erkennungsrate führen würde. Diese Annahme erwies sich jedoch als nicht vollständig korrekt: Tag-Familien mit mehr Tags (wie `tag36h11`) verfügen über eine bessere Fehlerkorrektur, was die Erkennung bei suboptimalen Bedingungen verbessern kann.

Ein weiterer Faktor war die gewünschte Tag-Größe: Um von der FHD-Kamera gut identifiziert zu werden, sollten die Tags möglichst groß auf den Streckenteilen angebracht werden. Die `tag16h5`-Familie ermöglicht größere Tags bei gleichem Druckbereich.

Im Projektverlauf wurde erkannt, dass ein Wechsel zu einer anderen Tag-Familie die Erkennungsprobleme möglicherweise verbessern könnte. Aufgrund von Zeitrestriktionen wurde diese Änderung jedoch nicht mehr umgesetzt und verbleibt als potenzielle Verbesserung für zukünftige Arbeiten.

4.3.4.2 Stabilität der Erkennung

Ein wichtiger Aspekt ist die Stabilität der Erkennung: Um ein „Flackern“ zu vermeiden (Tags werden kurzzeitig nicht erkannt und dann wieder erkannt), muss jeder Tag für mindestens **15 aufeinanderfolgende Frames** stabil erkannt werden (`STABLE_TAG_FRAMES = 15` in der Konfiguration). Erst dann wird er als gültiger Tag akzeptiert.

4.4 Crash-Detection

Um Abflüge und Stillstände des Fahrzeugs auf der realen Carrera-Bahn zu erkennen, war ein System notwendig, das per Kamera eine solche Situation

erkennt und weitergibt, aus diesem Grund wurde ein Skript zu Crash-Detection implementiert (crash_detection.py).

Die Crash Detection basiert auf der kontinuierlichen Auswertung der vom Kamerasystem gelieferten Positionsdaten des Fahrzeugs. Aus diesen Daten werden mehrere Indikatoren („Cues“) berechnet, die auf einen möglichen Crash hindeuten können. Erst wenn eine ausreichende Anzahl (Die Anzahl kann per „CRASH_CUE_COUNT“ kalibriert werden) dieser Hinweise über mehrere aufeinanderfolgende Frames (Die Anzahl kann per „N_CRASH_FRAMES“ kalibriert werden) hinweg vorliegt, wird ein Crash als bestätigt angesehen.

Aktuell sind die folgenden Cues implementiert, die Threshholds der Cues können individuell über die jeweiligen Variablen angepasst werden:

- Aus der Positionsänderung zwischen zwei Zeitpunkten wird die Geschwindigkeit des Fahrzeugs berechnet. Fällt diese unter einen definierten Schwellwert, deutet dies darauf hin, dass das Fahrzeug abrupt gestoppt ist, was häufig auf einen Abflug oder ein Blockieren hindeutet. Dies löst den Geschwindigkeits-Cue „cue_v“ aus. Um Messrauschen zu reduzieren, wird die Geschwindigkeit mithilfe von einer Double-Ended Queue „buffer_v“ über mehrere Frames hinweg geglättet.
- Zusätzlich wird geprüft, wie stark die tatsächliche Fahrzeugposition von einer vorhergesagten Position abweicht. Die Vorhersage basiert auf der zuletzt gemessenen Geschwindigkeit. Eine starke Abweichung kann auf einen plötzlichen Richtungswechsel oder ein Verlassen der Strecke hindeuten. Dies löst den Deviation-Cue „cue_dev“ aus.
- Ist die grüne Fläche auf dem Dach des Fahrzeugs für einen Frame ungetrackt, wird dies als weiteres Indiz für einen Crash gewertet. Dies tritt typischerweise auf, wenn es Fehler bei dem Tracking oder einen schweren Crash gibt, bei dem das Auto aus dem Bild fliegt oder auf dem Dach liegt. Dies löst den Area-Cue „cue_area“ aus.

Um Fehler zu vermeiden muss wie oben bereits erwähnt ein Crash für mehrere Frames in Folge erkannt werden, dies ist über eine Double-Ended Queue realisiert. So wird zu Beginn der Initialisierung eine Queue der Länge „N_CRASH_FRAMES“ erstellt. In jedem Frame, indem dann die Crash Detection aufgerufen wird, wird in diese Queue eingetragen, ob gemäß der Crash-Cues ein „Crash-Frame“ vorliegt oder nicht. Falls die komplette Queue mit „true“ gefüllt

ist, gibt die Crash Detection einen Crash aus. Das funktioniert, da ein richtiger Crash dafür sorgt, dass alle Folgeframes Crashframes sein werden.

4.5 Konfiguration

Die Feinabstimmung des Systems geschieht in der **config.py**, da die Änderungen die hier gemacht werden keinen Eingriff in den Quellcode erfordern.

4.5.1 Kamera-Setup

Durch den Befehl **CAM_INDEX** wird die Videoquelle gewählt. Diese muss man manuell anpassen.

4.5.2 Farbkalibrierung

Je nach den Lichtverhältnissen im Raum wird die Farberkennung beeinflusst. Die Schwellenwerte müssen unter Umständen angepasst werden, je nach Umgebung. Mithilfe eines dedizierten Kalibrierungsprozesses können über OpenCV verschiedene Masken und HSV-Schwellwerte bestimmt werden, die eine zuverlässige Unterscheidung zwischen Fahrzeugmarkierung und Hintergrund ermöglichen. Dieser Prozess ist besonders wichtig, da das Tracking-System bei extremen Lichtverhältnissen erhebliche Probleme hat: Zu helles Licht kann zu Überbelichtung führen, während zu dunkles Licht die Konturen verschwimmen lässt. Ein interessantes Problem, das im Projektverlauf auftrat, waren Fehlerkennungen in Texturen – beispielsweise wurde die Maserung von Holzböden gelegentlich fälschlicherweise als AprilTag interpretiert.

4.5.3 Kalibrierung der Crash-Detection-Parameter

Die Parameter für die Crash-Detection wurden durch einen iterativen Prozess ermittelt. Dabei war das Ziel, eine Balance zu finden zwischen zu empfindlicher Erkennung (Fehlalarme bei normalem Fahrverhalten) und zu unempfindlicher Erkennung (echte Crashes werden nicht erkannt).

Initialer Ansatz: Die Schwellwerte `V_THRESHOLD = 30.0` (Geschwindigkeitsschwelle) und `N_CRASH_FRAMES = 10` (benötigte aufeinanderfolgende Crash-Frames) wurden zunächst durch plausible Schätzungen festgelegt. Diese Schätzungen basierten auf Überlegungen wie: „Wenn das Auto langsamer als X Pixel pro Frame ist, steht es wahrscheinlich still."

Iterative Verfeinerung: Die initialen Schätzwerte wurden dann durch systematische Tests mit trainierten Modellen angepasst. Dabei wurde beobachtet:

- Bei zu niedrigem `V_THRESHOLD`: Langsames Fahren in Kurven wurde als Crash interpretiert
- Bei zu hohem `V_THRESHOLD`: Echte Stillstände nach Abflügen wurden nicht erkannt
- Bei zu niedrigem `N_CRASH_FRAMES`: Kurze Tracking-Aussetzer lösten Fehlalarme aus
- Bei zu hohem `N_CRASH_FRAMES`: Das System reagierte zu langsam auf echte Crashes

Die finalen Werte stellen einen empirisch ermittelten Kompromiss dar, der in der Praxis zuverlässig funktioniert.

4.5.4 Simulation & Netzwerk

Die Positionsdaten werden per TCP/IP an die Simulationsumgebung gesendet. Die Zieladresse wird durch die Konstante `SIMULATION_IP` (Standard: `127.0.0.1`) und den Port `SIMULATION_PORT` (Standard: `11360`) definiert. Die Sampling-Rate für die Fahrzeugposition beträgt 5 Hz (`SIMULATION_CAR_SAMPLING_RATE = 5.0`).

4.6 Integration & Datenfluss

1. `camera_manager.initialize_camera()` öffnet den Kamera-Stream mit plattformspezifischem Backend und setzt Belichtungsparameter.
2. Das Hauptprogramm (`main.py`) liest Frames und übergibt sie an den `CarTracker`.
3. `find_car()` liefert die X/Y-Koordinaten des Fahrzeugzentrums mittels HSV-Farberkennung.
4. Die AprilTag-Erkennung ordnet die Position einem Streckenabschnitt zu und berechnet den Fortschritt entlang der Strecke.
5. Position und Fortschritt werden über das `Network`-Modul als JSON an den Simulator gesendet.
6. Der Simulator antwortet mit Throttle-Werten, die an den Arduino weitergeleitet werden.
7. Falls `TRACK_VISUALIZATION_ENABLED` aktiv ist, werden Overlays auf das Live-Bild gezeichnet.

4.7 Auto-Reset-System

Nach einem erkannten Crash wird das Auto-Reset-System (`auto_reset.py`) aktiviert, um das Fahrzeug automatisch zur Startposition zurückzuführen.

4.7.1 Zustandsmaschine

Das System durchläuft folgende Zustände:

- **RUNNING**: Normaler Betrieb, RL-Agent steuert das Fahrzeug
- **WAITING_PLACEMENT**: Wartet auf manuelle Platzierung des Fahrzeugs auf der Strecke nach einem Crash
- **RETURNING_TO_START**: Fährt das Fahrzeug automatisch zur Startlinie zurück
- **WAITING_AT_START**: Kurze Pause an der Startposition (`AUTO_RESET_WAIT_AT_START_DURATION = 1.5s`)
- **COMPLETE**: Reset abgeschlossen, Episode kann neu starten

4.7.2 Geschwindigkeitssteuerung während Reset

Je nach Fortschritt auf der Strecke werden unterschiedliche Throttle-Werte verwendet:

- Unter 50%: Volle Geschwindigkeit (`AUTO_RESET_THROTTLES[0] = 40`)
- 50-80%: Mittlere Geschwindigkeit (`AUTO_RESET_THROTTLES[1] = 30`)
- Über 80%: Langsame Annäherung (`AUTO_RESET_THROTTLES[2] = 25`)

4.7.3 Grace Period

Nach einem abgeschlossenen Reset wird die Crash-Detection für eine definierte Anzahl an Frames deaktiviert (`AUTO_RESET_GRACE_PERIOD = 30` Frames), um Fehlerkennungen beim Anfahren zu vermeiden.

5 Simulator

Für das Training eines Reinforcement-Learning-Agenten ist eine große Anzahl an Trainingsdurchläufen erforderlich. Da wir uns das vermehrte wiederaufsetzen auf die Bahn aus Zeitgründen ersparen wollten, haben wir uns als Team dazu entschieden den frühen Lernprozess per Simulator zu realisieren.

5.1 Simulationsumgebung

5.1.1 Wahl der Game Engine

Als Simulationsumgebung wurde die Open-Source-Game-Engine Godot 4.5 gewählt. Diese Entscheidung fiel nach sorgfältiger Abwägung verschiedener Alternativen, wobei mehrere Faktoren den Ausschlag gaben:

Open-Source-Lizenz: Godot ist vollständig quelloffen und kostenlos nutzbar, ohne Lizenzgebühren oder Einschränkungen bei der Veröffentlichung. Dies steht im Gegensatz zu kommerziellen Engines wie Unity, die je nach Nutzungsumfang Lizenzkosten verursachen können.

Geringer Ressourcenverbrauch: Die Engine ist vergleichsweise leichtgewichtig und benötigt weniger Speicher und Rechenleistung als schwergewichtige Alternativen wie Unreal Engine. Dies ist besonders relevant für das parallele Training mehrerer Umgebungsinstanzen.

Minimaler Overhead: Godot verzichtet auf unnötige Funktionen und Bibliotheken („Bloat“), die für das Projekt nicht benötigt werden. Der schlanke Aufbau ermöglicht eine fokussierte Entwicklung ohne Ablenkung durch überflüssige Features.

Benutzerfreundlichkeit: Die integrierte Skriptsprache GDScript ist leicht erlernbar und ermöglicht schnelle Prototypentwicklung. Die Dokumentation ist umfangreich und die Community aktiv.

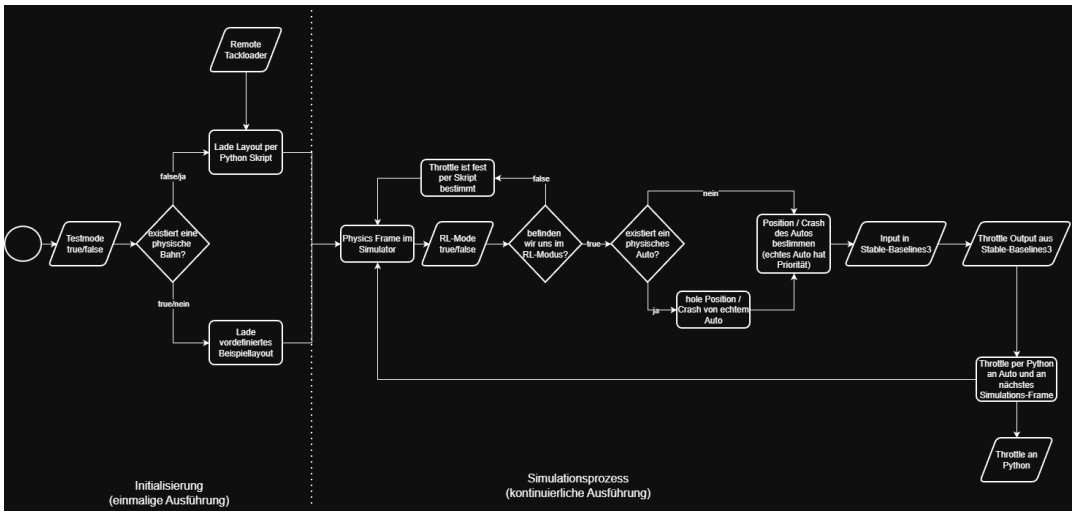
Erfahrungswerte: Teammitglieder hatten in der Vergangenheit negative Erfahrungen mit Unity gemacht, insbesondere hinsichtlich Performance-Problemen und der Komplexität des Lizenzmodells.

RL-Integration: Obwohl das Unity ML-Agents Toolkit technisch ausgereifter ist als das verwendete godot_rl_agents-Plugin, überwogen die anderen Vortei-

le von Godot. Die vorhandenen Einschränkungen des Godot-RL-Frameworks wurden durch eigene Anpassungen kompensiert.

Godot bietet eine flexible 3D-Umgebung, ein integriertes Physiksystem sowie die Möglichkeit, vorgefertigte RL-Frameworks wie das im Projekt eingesetzte Stable-Baselines3 einzubinden. Ein weiterer Vorteil liegt in der guten Balance zwischen physikalischer Genauigkeit und Rechenperformance, was insbesondere für das parallele oder beschleunigte Training von RL-Agenten relevant ist.

5.2 Simulationsablauf



Die Abbildung zeigt den Ablauf des Simulationsprozesses.

1. Zu Beginn wird die globale Variable **TEST_MODE** eingelesen. Anhand dieser Variable entscheidet das System, ob eine reale Strecke erkannt und geladen werden muss, oder ob man auf ein im Code vorgefertigtes Layout zurückfallen wird.
2. Ist **TEST_MODE** auf true gesetzt, wird ein hardcoded Streckenmodell geladen. Im Falle von false allerdings, wird der „remote_trackloader“ aufgerufen, der von einer Python Schnittstelle aus einer echten Strecke das entsprechende Modell übergibt (Mehr dazu im folgenden Abschnitt).

Punkt 1 & 2 werden pro Ausführung der Simulation nur jeweils einmal ausgeführt, während die kommenden Punkte in einer kontinuierlichen Schleife ausgeführt werden.

3. Zu Beginn des Loops wird überprüft, ob sich das Auto im „RL_MODE“ befindet. Dieser Modus ist für uns zum Debuggen vorgesehen und sorgt dafür, dass sich das simulierte Auto nicht per Reinforcement Learning, sondern per fest im Code vorgegeben Werten Werten fortbewegt.
4. Ist **RL_MODE** auf false, wird eben dieser Wert gewählt und die Loop ist direkt ohne weiteres beendet, wie bereits erwähnt ist dies nur zum Debuggen vorgesehen.
5. Ist **RL_MODE** auf true, kann die Loop fortfahren und es wird als nächstes überprüft, ob es ein reales Auto gibt, welches berücksichtigt werden soll.
6. Falls vorhanden wird die Position des realen Autos per Python Skript abgefragt und auf die Simulation übertragen (Näheres zur Berücksichtigung der Abweichungen hier).
7. Die Position des Autos (sowie potentielle Crashes) werden dann als Input in das Stable-Baselines3 Framework gegeben, aus welchem wir daraufhin den nächsten Throttle abrufen können.
8. Dieser Throttle wird dann zum einen per Python Skript an die Bahn gegeben, um das potentielle reale Auto anzusteuern. Zum Anderen wird der Throttle in der Simulation direkt im nächsten „Physics Frame“ genutzt, um die Simulation laufen zu lassen. Damit ist die Schleife geschlossen und beginnt erneut von Schritt 3.

5.3 Modellierung der Carrera Strecke

5.3.1 Streckenteile

Um die Carrera-Bahn zu modellieren, wurden die einzelnen nutzbaren Carrera-Streckenteile modular in Godot nachgebaut. Dabei wurden folgende Streckentypen modelliert:

- **FINISH (ID 1)**: Start-/Zielgerade (kurze Gerade)
- **SHORT_STRAIGHT (ID 2)**: Kurze Gerade
- **MEDIUM_STRAIGHT (ID 3)**: Mittlere Gerade
- **LONG_STRAIGHT (ID 4)**: Lange Gerade
- **RIGHT_CURVE (ID 5)**: Rechtskurve
- **LEFT_CURVE (ID 6)**: Linkskurve

Die IDs entsprechen den AprilTag-Zuordnungen im Tracking-System. Durch die Modellierung dieser modularen Streckenteile ist es möglich, im Code durch eine Verkettung der einzelnen Elemente eine vollständige Strecke zu erzeugen.

Diese Verkettung kann entweder statisch als Array im Code definiert (z.B. `[1, 4, 5, 5, 6, 4, 6, 5, 5, 4, 2, 5, 4, 4, 4, 5]`) oder dynamisch per Skript aus einem Kamerafeed ausgelesen werden.

5.3.2 Import einer realen Strecke

Wie bereits in einem vorherigen Kapitel erläutert, erfolgt die Erkennung einer realen Strecke mithilfe von AprilTags. Da jedes Streckenteil eines bestimmten Typs mit demselben AprilTag versehen ist, lässt sich dieses Verfahren direkt auf den modularen Streckenaufbau übertragen.

Auf diese Weise können die erkannten AprilTags sequenziell eingelesen und jeweils einem entsprechenden Streckenabschnittstyp zugeordnet werden. Anschließend wird das passende Streckenmodell an den bestehenden Streckenaufbau angehängt. Dadurch ist es möglich, eine reale Carrera-Strecke automatisiert in den Simulator zu übertragen und dort für Training und Evaluation zu verwenden.

5.3.3 Flexibilität bei Streckenänderungen

Ein wichtiger Vorteil der modularen Architektur ist die einfache Anpassung an neue Streckenlayouts. Da das System bei jedem Start die Strecke automatisch einliest und an den Simulator übermittelt, ist der Wechsel zu einer anderen Streckenkonfiguration unkompliziert:

1. **Physischer Umbau**: Die Streckenteile werden nach Belieben neu arrangiert
2. **Automatische Erkennung**: Beim nächsten Systemstart erkennt die Kamera automatisch die neue Streckenreihenfolge via AprilTags
3. **Training**: Es muss lediglich ein neues Modell für die neue Strecke trainiert werden

Die einzige zeitliche Investition bei einem Streckenwechsel ist somit das Training des RL-Agenten auf der neuen Konfiguration. Das Tracking- und Kommunikationssystem selbst erfordert keine Anpassungen, was die praktische Nutzung erheblich vereinfacht.

5.4 Annäherung des Physikalischen Fahrzeugmodells

Das im Simulator verwendete Fahrzeugmodell stellt eine Annäherung an das reale Fahrverhalten eines Carrera-Fahrzeugs dar. Ziel war es, einen sinnvollen

Trade-off zwischen realistischem Verhalten und verhältnismäßiger Komplexität zu finden. Ein perfekt realistisches Modell wäre schwer zu erstellen und würde das Training unnötig verlangsamen, während ein zu vereinfachtes Modell dazu führen würde, dass das Gelernte nicht auf die reale Strecke übertragbar ist.

5.4.1 Bewegungsmodell

Die Bewegung des Fahrzeugs erfolgt entlang einer vordefinierten Streckenführung. In Godot wird dies durch eine `Curve3D`-Node realisiert, die den Mittelpfad der Strecke beschreibt. Das Fahrzeug folgt diesem Pfad und berechnet seine neue Position basierend auf der aktuellen Geschwindigkeit und der verstrichenen Zeit.

Die Fahrzeugsteuerung beschränkt sich auf die Regelung des Throttles (Gaspedal), was der realen Eingriffsmöglichkeit bei Carrera-Fahrzeugen entspricht – es gibt kein Lenkrad, nur mehr oder weniger Gas. Der gewählte Throttle-Wert (0 bis 1) bestimmt die Zielgeschwindigkeit des Fahrzeugs.

5.4.2 Beschleunigungsmodell

Im echten Leben kann ein Fahrzeug nicht instantan von 0 auf 100 beschleunigen oder umgekehrt. Um dies nachzubilden, werden Beschleunigungs- und Verzögerungsvorgänge über lineare Interpolation (`lerp`) modelliert:

- **Beschleunigung**: `THROTTLE_ACCEL = 20.0` – Wie schnell das Fahrzeug bei mehr Gas beschleunigt
- **Verzögerung**: `THROTTLE_DEACCEL = 5.0` – Wie schnell das Fahrzeug bei weniger Gas abbremst

Die asymmetrische Konfiguration (schnellere Beschleunigung als Verzögerung) spiegelt das Verhalten realer Slot-Cars wider: Sie beschleunigen relativ schnell, verlieren aber ihre Geschwindigkeit durch Reibung nur langsam.

5.4.3 Crash-Erkennung im Simulator (Side-Drag)

Eine der wichtigsten Funktionen des Simulators ist die Erkennung, wann ein Fahrzeug „abfliegen“ würde. In der Realität passiert das, wenn das Fahrzeug zu schnell in eine Kurve fährt und die Fliehkraft die Haftung übersteigt.

Im Simulator wird dies über das Konzept des **Side-Drags** modelliert. Der Side-Drag berechnet sich aus der Seitwärtsgeschwindigkeit des Fahrzeugs relativ zur aktuellen Streckenrichtung:

```
var sideways_speed = velocity.dot(right)
var side_drag = abs(sideways_speed)
```

Anschaulich erklärt: Wenn das Fahrzeug geradeaus fährt, ist der Side-Drag nahe Null. Wenn es jedoch versucht, mit hoher Geschwindigkeit abzubiegen, „möchte“ es eigentlich geradeaus weiterfahren (Trägheit), was sich in einer hohen seitlichen Kraft äußert.

5.4.3.1 Kalibrierung des Schwellwerts

Der Schwellwert `DRAG_SENSITIVITY = 15` wurde durch systematischen Vergleich zwischen Simulation und realer Strecke ermittelt. Das Vorgehen war wie folgt:

1. Auf der realen Strecke wurde gemessen, ab welcher Geschwindigkeit das Fahrzeug in verschiedenen Kurventypen die Haftung verliert und von der Strecke fliegt.
2. Die gleichen Szenarien wurden im Simulator nachgestellt.
3. Der `DRAG_SENSITIVITY`-Wert wurde iterativ angepasst, bis das simulierte Fahrzeug bei ähnlichen Geschwindigkeiten „abflog“ wie das reale.

Überschreitet der Side-Drag den Schwellwert für mindestens **4 aufeinanderfolgende Frames** (`DRAG_CRASH_FRAMES_NEEDED = 4`), gilt das Fahrzeug als abgeflogen. Die zeitliche Komponente verhindert Fehlerkennungen durch kurzzeitige Schwankungen.

5.4.4 Kalibrierung der Simulationsparameter

Die Übereinstimmung zwischen Simulation und Realität war ein zentrales Designziel. Die Parameter wurden so abgestimmt, dass das simulierte Fahrzeug leicht konservativer und vorsichtiger fährt als theoretisch möglich. Dieser bewusste Sicherheitspuffer lässt Spielraum für die unvermeidliche Varianz zwischen Simulation und Realität – Faktoren wie Reifenabnutzung, Temperaturunterschiede oder leichte Streckenunregelmäßigkeiten können das reale Fahrverhalten beeinflussen.

5.5 Machine Learning Framework

5.5.1 Warum Stable-Baselines3?

Für die Implementierung und das Training des Reinforcement-Learning-Agenten wurde das Framework Stable-Baselines3 (SB3) verwendet. SB3 ist eine

weit verbreitete, in Python implementierte Bibliothek für Deep Reinforcement Learning, die auf PyTorch basiert.

Die Wahl fiel auf SB3 aus mehreren Gründen:

- **Zuverlässigkeit**: SB3 wird aktiv gewartet und bietet gut getestete Implementierungen etablierter RL-Algorithmen
- **Dokumentation**: Umfangreiche Dokumentation und Beispiele erleichtern den Einstieg
- **Flexibilität**: Unterstützung für verschiedene Algorithmen (PPO, SAC, A2C, etc.) ermöglicht einfaches Experimentieren
- **Integration**: Gute Kompatibilität mit dem godot_rl_agents-Plugin

5.5.2 PPO-Konfiguration

Die Hyperparameter eines RL-Algorithmus haben erheblichen Einfluss auf das Lernverhalten. Die wichtigsten Parameter im Projekt:

- **Learning Rate**: 0.0003 (optional mit linearem Schedule) – Bestimmt, wie stark die Gewichte des neuronalen Netzwerks bei jedem Update angepasst werden. Ein zu hoher Wert führt zu instabilem Lernen, ein zu niedriger zu sehr langsamem Fortschritt.
- **Gamma (Discount-Faktor)**: 1.0 – Dieser Parameter bestimmt, wie stark zukünftige Belohnungen im Vergleich zu sofortigen Belohnungen gewichtet werden. Die Entscheidung, keine Diskontierung zu verwenden (Gamma = 1.0), wurde bewusst getroffen. Tests zeigten, dass ein Discount-Faktor < 1.0 keine signifikante Verbesserung brachte, da das Rundenfahren ein zyklisches Problem ist – jede Position auf der Strecke ist gleich wichtig, es gibt keinen „Endzustand“, der besonders belohnt werden sollte.
- **Entropy Coefficient**: 0.0001 – Fördert die Exploration, indem zufällige Aktionen leicht belohnt werden. Dies verhindert, dass der Agent zu früh in einem suboptimalen Verhalten „stecken bleibt“.
- **n_steps**: 32 – Die Anzahl der Schritte, die gesammelt werden, bevor ein Policy-Update durchgeführt wird.
- **Policy**: MultiInputPolicy – Ermöglicht die Verarbeitung strukturierter Beobachtungen (mehrere Eingabewerte statt einem einzelnen Vektor).

5.5.3 Integration mit Godot

Die technische Kopplung zwischen SB3 (Python) und Godot (Game Engine) erfolgt über das `godot_rl_agents`-Addon. Dieses Plugin ermöglicht bidirektionale Kommunikation über Sockets: Godot sendet Beobachtungen und Belohnungen, Python sendet Aktionen zurück.

Der Trainingsloop (`stable_baselines3_example.py`) bietet verschiedene Optionen:

- **Paralleles Training** (`--n_parallel`): Mehrere Umgebungsinstanzen laufen gleichzeitig, was das Training deutlich beschleunigt
- **Beschleunigtes Training** (`--speedup`): Die Simulation läuft schneller als Echtzeit
- **Checkpoint-Speicherung** (`--save_checkpoint_frequency`): Regelmäßiges Speichern des Modells ermöglicht Wiederaufnahme bei Unterbrechung
- **ONNX-Export** (`--onnx_export_path`): Exportiert das trainierte Modell in ein portables Format für Inferenz

5.6 Rolle des Simulators im Gesamtsystem

5.6.1 Betriebsmodi

Der Simulator unterstützt zwei Hauptmodi:

- **SIMULATION_MODE = true**: Reines Simulationstraining ohne reale Hardware
- **SIMULATION_MODE = false**: Hybridmodus mit Synchronisation zur realen Strecke

5.6.2 Typischer Trainingsablauf

1. **Simulationstraining**: 30.000+ Schritte im beschleunigten Modus (Speedup 32x, 8 parallele Instanzen), was etwa 1-2 Stunden Trainingszeit entspricht
2. **Hybridtraining**: 1.000 Schritte mit Synchronisation zur realen Strecke
3. **Evaluation**: Inferenz auf der realen Strecke mit trainiertem Modell

Die Kombination aus 32-facher Beschleunigung und 8 parallelen Umgebungen ermöglicht es, Millionen von Simulationsschritten in wenigen Stunden durchzuführen.

5.6.3 Launcher-System

Das Projekt enthält einen Launcher (`launcher/launcher.py`), der verschiedene Komponenten koordiniert und die manuelle Ausführung mehrerer Prozesse überflüssig macht. Der Launcher wird über Profile in einer YAML-Konfigurationsdatei (`config.yaml`) gesteuert:

5.6.3.1 Verfügbare Profile

- **full**: Startet Godot-Simulator + SB3-Training + Python-Tracking (komplettes System)
- **training**: Startet nur Godot + Training (für reines Simulationstraining)
- **sb-full-train**: Vollständiges Training mit Export-Binary (für Server ohne GUI)
- **python**: Nur Python-Tracking-System starten (für Tests oder Hybrid-Betrieb)

5.6.3.2 Typischer Workflow

```
python launcher/launcher.py --profile=full --timesteps=100000
```

Der Launcher sorgt automatisch für:

- Korrekte Startreihenfolge der Prozesse (Godot vor Training)
- Umgebungsvariablen und Arbeitsverzeichnisse
- Logging der Ausgaben aller Komponenten
- Graceful Shutdown aller Prozesse bei Beendigung

6 Reinforcement Learning Prozess

6.1 Aktions- & Zustandsraum

6.1.1 Aktionsraum

Der Aktionsraum bezieht sich auf die Steuerung des Fahrzeugs über Throttle-Werte. Die aktuelle Implementierung verwendet einen **kontinuierlichen Aktionsraum** mit einem einzelnen Wert zwischen 0 und 1:

```
func get_action_space() -> Dictionary:
    return {"throttle": {"size": 1, "action_type": "continuous"}}
```

Der vom RL-Agenten gewählte Throttle-Wert wird anschließend auf den Bereich 0-100 skaliert und an den Arduino gesendet.

Hinweis: In früheren Projektphasen wurden auch diskrete Aktionsräume mit fünf Stufen (0, 25, 50, 75, 100) evaluiert. Dieser Ansatz wurde verworfen, da das Modell dazu neigte, sich auf sichere mittlere Werte festzulegen, anstatt dynamisch zwischen hohem und niedrigem Throttle zu wechseln.

6.1.2 Zustandsraum

Der Zustandsraum definiert, welche Informationen der Agent über seine Umgebung erhält. Die Auswahl der richtigen Observationen ist entscheidend: Zu wenig Information macht es dem Agenten unmöglich, gute Entscheidungen zu treffen; zu viel Information verlangsamt das Lernen und kann zu Overfitting führen.

Der aktuelle Zustandsraum enthält drei diskretisierte Observationen:

```
func get_obs() -> Dictionary:
    return {"obs": [
        racecar.discrete_progress_ratio,  // Fortschritt auf der
Strecke (%)
        racecar.is_out,                    // Crash-Status
(Boolean)
        racecar.discrete_velocity          // Geschwindigkeit
(diskretisiert)
    ]}
```

Fortschritt (progress_ratio): Gibt an, wie weit das Fahrzeug auf der Strecke ist (0% = Start, 100% = Ziel). Diese Information ist wichtig, da der Agent lernen

muss, dass bestimmte Streckenabschnitte (Kurven) eine andere Strategie erfordern als andere (Geraden).

Crash-Status (is_out): Ein Boolean, der anzeigt, ob das Fahrzeug gerade als „abgeflogen“ gilt. Dies ermöglicht dem Agenten, auf Crash-Situationen zu reagieren.

Geschwindigkeit (velocity): Die aktuelle Geschwindigkeit des Fahrzeugs. Wichtig, da der Agent wissen muss, wie schnell er aktuell fährt, um zu entscheiden, ob er beschleunigen oder bremsen sollte.

Die Diskretisierung erfolgt mit konfigurierbaren Schrittgrößen:

- **discrete_progress_ratio_step_size**: 5.0% – Der Fortschritt wird in 20 Stufen eingeteilt
- **discrete_velocity_step_size**: 2.0 Einheiten – Ähnliche Geschwindigkeiten werden zusammengefasst

Die Diskretisierung reduziert die Komplexität des Zustandsraums und kann das Lernen beschleunigen.

6.2 Reward-Funktion

Die Reward-Funktion ist das Herzstück des Reinforcement Learning. Sie definiert, was „gutes“ und was „schlechtes“ Verhalten ist. Die Gestaltung dieser Funktion (oft „Reward Engineering“ genannt) ist eine der schwierigsten Aufgaben bei RL-Projekten und erfordert in der Praxis häufig mehrere Iterationen.

6.2.1 Entwicklung der Reward-Struktur

Die finale Reward-Struktur ist das Ergebnis eines iterativen Prozesses. Das grundlegende Vorgehen war Trial-and-Error: Verschiedene Konfigurationen wurden im Simulator getestet und anhand zweier Hauptkriterien bewertet:

1. Fährt das Fahrzeug schneller als zuvor?
2. Wie oft fliegt es von der Strecke?

Die Grundidee orientiert sich an etablierter Literatur zum Thema Reinforcement Learning für Fahrzeugsteuerung: Rewards sollten proportional zur Geschwindigkeit sein, Crashes sollten ausreichend bestraft werden, und die Simulationsparameter sollten möglichst nah an der realen Welt liegen.

Die Reward-Funktion (`give_reward()` in `racecar_base.gd`) kombiniert mehrere Anreize, um ein ausgewogenes Lernziel zu schaffen:

6.2.2 Kontinuierliche Throttle-Belohnungen

Pro Physics-Frame werden je nach aktuellem Throttle-Wert Belohnungen oder Bestrafungen vergeben. Das Ziel ist, den Agenten zu ermutigen, so schnell wie möglich zu fahren:

- `throttle >= 0.75`: +0.001 × delta (Belohnung für hohe Geschwindigkeit)
- `throttle >= 0.6`: −0.005 × delta (leichte Strafe)
- `throttle >= 0.5`: −0.01 × delta (moderate Strafe)
- `throttle >= 0.45`: −0.02 × delta (stärkere Strafe)
- Sonst: −0.03 × delta (maximale Strafe für zu langsames Fahren)

6.2.2.1 Wahl der Schwellwerte

Die spezifischen Schwellwerte (0.75, 0.6, 0.5, 0.45) wurden durch mehrere Faktoren bestimmt:

1. **Übernahme bewährter Werte**: Die Grundstruktur basiert auf Parametern einer früheren Projektphase, die sich als funktional erwiesen hatten.
2. **Fokus auf den kritischen Bereich**: Der Bereich zwischen 0.45 und 0.75 entspricht dem Geschwindigkeitsbereich, in dem die Bahn am ehesten zu steuern ist und die meiste Präzision erfordert. Bei niedrigeren Werten ist das Fahrzeug zu langsam für interessante Dynamik, bei höheren Werten wird das Risiko eines Crashs dominant.
3. **Ursprüngliche Struktur**: Eine frühere Version der Reward-Funktion hatte eine feinere Staffelung (ähnlich der Literatur), jedoch wurden viele Werte in der Praxis nie erreicht. Die aktuelle Struktur konzentriert die Differenzierung auf den relevanten Bereich und ermöglicht dadurch mehr Präzision mit weniger Parametern.

Diese Staffelung sorgt dafür, dass der Agent nicht einfach „auf Nummer sicher geht“, indem er langsam fährt. Langsames Fahren wird bestraft, schnelles Fahren belohnt.

6.2.3 Rundenbasierte Belohnungen

Zusätzlich zu den kontinuierlichen Belohnungen gibt es Bonuspunkte für Meilensteine:

- **Runde geschafft**: +0.1 (Episode wird beendet)
- **Runde geschafft mit neuer Bestzeit**: +10.0 – Ein großer Bonus motiviert den Agenten, nicht nur sicher, sondern auch schnell zu fahren

- **Leicht verbesserte Zeit**: +1.0 Bonus

6.2.4 Crash-Bestrafung

Abstürze werden hart bestraft, um dem Agenten beizubringen, Risiken zu managen:

- **Crash erkannt (Side-Drag im Simulator)**: −5.0 (Episode wird beendet)
- **Kamera-Crash erkannt (auf realer Strecke)**: −10.0

6.2.4.1 Begründung der asymmetrischen Bestrafung

Die höhere Bestrafung für Kamera-Crashes (-10 vs. −5) wurde bewusst gewählt:

1. **Praktische Konsequenzen**: Ein Crash auf der realen Strecke erfordert manuelle Intervention – das Fahrzeug muss physisch wieder aufgesetzt werden, was Zeit kostet und den Trainingsfluss unterbricht.
2. **Potenzielle Schäden**: Bei einem echten Crash besteht die Möglichkeit, dass das Fahrzeug oder die Strecke beschädigt werden.
3. **Konservatives Verhalten fördern**: Wenn das Modell auf der realen Strecke trainiert wird, soll es lernen, sicherer zu fahren und weniger zu experimentieren als in der Simulation.
4. **Zeiteffizienz**: Weniger Crashes bedeuten mehr produktive Trainingszeit auf der realen Strecke.

6.3 Trainingsablauf

Der typische Trainingsablauf gliedert sich in drei Phasen, die aufeinander aufbauen:

6.3.1 Phase 1: Simulationstraining

In der ersten Phase wird das Modell vollständig in der Simulation trainiert. Dies hat mehrere Vorteile:

- Keine Gefahr für die Hardware
- Beschleunigtes Training (32x schneller als Echtzeit)
- Parallele Ausführung (8 Instanzen gleichzeitig)

```
python stable_baselines3_example.py --timesteps=500_000 --
speedup=32 --n_parallel=8
```

In dieser Phase lernt das Modell die grundlegende Streckennavigation: Wann muss gebremst werden? Wie schnell kann auf Geraden gefahren werden?

6.3.2 Phase 2: Feintuning mit realer Strecke

Nach dem Simulationstraining wird das Modell mit der realen Strecke konfrontiert:

- Die Strecke wird per AprilTags eingelesen und in den Simulator übertragen
- Das vortrainierte Modell wird mit realen Positionsdaten nachtrainiert
- Im Hybridmodus synchronisiert sich die Simulation mit der tatsächlichen Fahrzeugposition

Diese Phase ist wichtig, um den „Sim-to-Real Gap“ zu überbrücken – die Unterschiede zwischen Simulation und Realität.

6.3.3 Phase 3: Evaluation

In der letzten Phase wird das trainierte Modell auf der realen Strecke getestet:

```
python stable_baselines3_example.py --inference --
resume_model_path=model.zip
```

Hier zeigt sich, ob das Training erfolgreich war. TensorBoard ermöglicht die Visualisierung und Analyse der Ergebnisse.

7 Herausforderungen

7.1 Lichtverhältnisse

Im Laufe des Projektes waren die Lichtverhältnisse eine große Herausforderung. Durch suboptimale & ungleichmäßige Beleuchtung war das Kameratracking inkonsistent. So wurden die AprilTags und das Auto nicht immer alle erkannt und das System konnte so nicht einwandfrei funktionieren. Als Lösung wurde eine Softbox besorgt, die für eine gleichmäßigere Beleuchtung sorgt. Zudem wurden die Kameratracking- und Farbbereiche dahingehend angepasst, dass die Objekte stabiler erkannt werden.

7.2 Kameraqualität

Eine weitere wesentliche Herausforderung im Projekt stellte die Qualität der verwendeten Kamera dar. Insbesondere in Kombination mit wechselnden Lichtverhältnissen erwies sich das Kamerabild häufig als unscharf. Diese Unschärfe hatte direkte Auswirkungen auf die Zuverlässigkeit der visuellen Wahrnehmung und erschwerte mehrere nachgelagerte Verarbeitungsschritte. Insbesondere das Erkennen der AprilTags, sowie das konstante Tracking der grünen Fläche des Autos erwies sich dadurch als herausfordernd.

Zusätzlich war die Bildrate der Kamera vergleichsweise niedrig. Die geringe FPS-Zahl hatte zur Folge, dass schnelle Bewegungen des Fahrzeugs nur in groben zeitlichen Abständen erfasst wurden. Gerade bei höheren Geschwindigkeiten fehlten dadurch Zwischenzustände, was sowohl die Positionsschätzung als auch die zeitnahe Reaktion des Systems erschwerte. In der Praxis führte dies zu einer erhöhten Verzögerung zwischen tatsächlicher Fahrzeugbewegung und wahrgenommener Position.

Diese Probleme konnten durch langwieriges Troubleshooten und adjustieren der Kameraposition, sowie -optionen gelöst werden.

8 Evaluation

Die Evaluation des trainierten Modells erfolgte in mehreren Schritten, wobei sowohl quantitative Metriken als auch qualitatives Fahrverhalten bewertet wurden.

8.1 Trainingsmetriken

Die Entwicklung des Agenten wurde über TensorBoard verfolgt. Dabei wurden folgende Metriken überwacht:

- **Episode Reward Mean**: Durchschnittliche kumulative Belohnung pro Episode
- **Episode Length Mean**: Durchschnittliche Länge einer Episode (in Schritten)
- **Value Loss**: Verlust der Value-Function-Approximation
- **Policy Loss**: Verlust der Policy-Optimierung

Nach ca. 30.000 Schritten im beschleunigten Simulationsmodus (32x Speedup, 8 parallele Instanzen) zeigte sich eine deutliche Konvergenz des Reward-Signals.

8.1.1 Trainingszeiten im Detail

Die Trainingszeit variiert erheblich je nach Konfiguration des Aktionsraums:

Diskreter Aktionsraum (5 Werte: 0, 25, 50, 75, 100):

- Bei einem Speedup von 32x und einer einzelnen Umgebung
- Konvergenz nach ca. 10.000 Schritten
- Trainingszeit: ca. 10 Minuten

Kontinuierlicher Aktionsraum:

- Bei einem Speedup von 32x und 8 parallelen Umgebungen
- Konvergenz nach ca. 1.000.000 Schritten
- Trainingszeit: ca. 40 Minuten

Der deutlich höhere Trainingsaufwand beim kontinuierlichen Aktionsraum resultiert aus der größeren Komplexität: Statt aus 5 diskreten Optionen zu wählen, muss der Agent lernen, einen beliebigen Wert im Intervall $[0, 1]$ zu wählen. Diese zusätzliche Freiheit führt zu besseren Ergebnissen, erfordert aber mehr Exploration.

8.2 Qualitative Bewertung

Das trainierte Modell zeigt folgendes Verhalten:

- **Geraden**: Hoher Throttle (80-100%), maximale Geschwindigkeit
- **Kurvenannäherung**: Progressives Abbremsen vor Kurven
- **Kurvenfahrt**: Moderater Throttle (40-60%), um Side-Drag unter dem Schwellwert zu halten
- **Kurvenausgang**: Schnelle Beschleunigung nach der Kurve

8.2.1 Erreichte Performance

Beste Rundenzeit auf realer Strecke: ca. 2,9 Sekunden

Diese Zeit ist von einem geübten menschlichen Fahrer schlagbar, liegt jedoch deutlich über dem Niveau eines Anfängers. Das trainierte Modell erreicht eine gute Balance zwischen Geschwindigkeit und Sicherheit.

Erfolgsrate: Bei einem vollständig trainierten Modell traten gegen Ende der Entwicklung nahezu keine Crashes mehr auf. Das Modell hatte gelernt, die Grenze zwischen maximal möglicher Geschwindigkeit und Stabilitätsverlust zuverlässig einzuschätzen.

8.3 Sim-to-Real Transfer

Der Transfer des in der Simulation trainierten Modells auf die reale Strecke war der kritischste Teil der Evaluation. Die Herausforderungen dabei:

- Unterschiede in der Physik (Reifenhaftung, Beschleunigung)
- Sensorrauschen der Kamera
- Latenz der Kommunikationskette

8.3.1 Umgang mit dem Sim-to-Real Gap

Gegen Ende des Projekts waren die Simulationswerte relativ gut abgestimmt. Die bewusste Entscheidung, den Simulator leicht konservativer einzustellen als theoretisch notwendig, zahlte sich aus: Der Sicherheitspuffer kompensierte die unvermeidlichen Unterschiede zwischen Simulation und Realität, wie Reifenabnutzung, Temperaturunterschiede oder leichte Streckenunregelmäßigkeiten.

Das Modell zeigte nach Feintuning mit realen Daten akzeptables Fahrverhalten, wobei die Rundenzeiten nicht die theoretisch optimalen Werte erreichten – ein bewusster Kompromiss zugunsten der Zuverlässigkeit.

9 Projektorganisation

9.1 Teamstruktur und Arbeitsmethodik

Das Projekt wurde nach dem SCRUM-Framework organisiert, einer agilen Methodik, die sich durch iterative Entwicklung und regelmäßige Anpassung an neue Erkenntnisse auszeichnet. Diese Herangehensweise erwies sich als besonders geeignet für ein Forschungsprojekt, bei dem viele Parameter erst durch Experimente ermittelt werden konnten.

9.1.1 Aufgabenverteilung

Die Aufgaben wurden nach SCRUM-Prinzipien verteilt, wobei die Teammitglieder je nach Expertise und Verfügbarkeit verschiedene Komponenten übernahmen. Die modulare Architektur des Systems (Python-Tracking, Godot-Simulator, Arduino-Hardware) ermöglichte parallele Entwicklung an verschiedenen Teilsystemen.

9.1.2 Zeitplan und Meilensteine

Der Projektzeitplan orientierte sich an den Prüfungs- und Abgabeterminen des Studiums. Diese externen Deadlines fungierten als feste Meilensteine, an denen jeweils ein funktionsfähiger Stand des Systems demonstriert werden musste.

9.1.3 Kommunikation

Die Teamkommunikation erfolgte über mehrere Kanäle:

- **Wöchentliche Präsenzmeetings**: Regelmäßiger Austausch im Team, Abstimmung der nächsten Schritte und Diskussion aufgetretener Probleme
- **Wöchentliche Meetings mit dem Betreuer**: Feedback zum Projektfortschritt und fachliche Unterstützung durch Herrn Radke
- **Flexible Arbeitszeiten**: Neben den festen Terminen wurde hauptsächlich mittwochs und donnerstags gemeinsam gearbeitet

10 Fazit und Ausblick

10.1 Fazit

Das Projekt demonstriert erfolgreich die Anwendung von Reinforcement Learning zur autonomen Steuerung eines Carrera-Fahrzeugs. Die Kombination aus Simulationstraining und realem Hardware-Transfer erwies sich als praktikable Strategie, um den Trainingsaufwand zu reduzieren und gleichzeitig praxistaugliche Ergebnisse zu erzielen.

Die wesentlichen Erkenntnisse:

- **PPO als geeigneter Algorithmus**: Die Stabilität und Robustheit von PPO erwies sich als Vorteil für das rauschbehaftete Szenario. Andere Algorithmen wie DQN, SAC oder TD3 wurden zwar diskutiert, aber aufgrund von Zeitbeschränkungen nicht evaluiert.
- **Kontinuierlicher Aktionsraum**: Der Wechsel von diskreten zu kontinuierlichen Throttle-Werten verbesserte das Lernverhalten signifikant. Bei diskreten Werten neigte das Modell dazu, sichere Mittelwerte zu wählen.
- **Crash-Detection**: Die Multi-Cue-Erkennung (Geschwindigkeit, Abweichung, Fläche) ermöglicht zuverlässige Crash-Erkennung, wobei die Parameterwerte durch iteratives Testen kalibriert wurden.
- **Modularität**: Die Trennung in Python-Tracking, Godot-Simulator und Arduino-Steuerung ermöglicht unabhängige Entwicklung und Tests.
- **Sim-to-Real Transfer**: Durch konservative Abstimmung der Simulationsparameter konnte der Transfer auf die reale Strecke mit akzeptablen Ergebnissen realisiert werden.

10.2 Limitationen

Während des Projekts wurden folgende Einschränkungen identifiziert:

- **Hardcoded Track Order**: Die automatische Streckenerkennung wird aktuell durch fest eingetragene Werte überschrieben
- **Feste Farbkalibrierung**: Die HSV-Schwellwerte müssen bei geänderten Lichtverhältnissen manuell angepasst werden
- **Keine Multi-Car-Unterstützung**: Das System erkennt aktuell nur ein einzelnes Fahrzeug

- **Lichtempfindlichkeit**: Das Tracking-System hat bei extremen Lichtverhältnissen (zu hell oder zu dunkel) erhebliche Probleme bei der Erkennung der AprilTags. Trotz mehrerer Iterationen zur Optimierung der OpenCV-Masken werden gelegentlich falsche Tags in Texturen erkannt (z.B. in der Holzmaserung des Bodens).
- **Fehlende Fallback-Mechanismen**: Wenn die Kamera während des Betriebs ausfällt oder das Bild schwarz wird, kann das Modell nicht mehr korrekt fahren. Es gibt keinen automatischen Fallback.

10.3 Ausblick

Für zukünftige Arbeiten bieten sich folgende Erweiterungen an:

10.3.1 Kurzfristig (Bug-Fixes)

- Entfernung der hardcoded Track Order und Nutzung der berechneten Werte
- Implementierung einer automatischen Farbkalibrierung
- Einführung eines strukturierten Logging-Systems
- **Wechsel zu einer robusteren AprilTag-Familie**: Der Wechsel von tag16h5 zu tag36h11 könnte die Erkennungsprobleme bei suboptimalen Bedingungen verbessern. Die tag36h11-Familie verfügt über eine bessere Fehlerkorrektur, was insbesondere bei schwierigen Lichtverhältnissen oder teilweise verdeckten Tags von Vorteil wäre.

10.3.2 Mittelfristig (Features)

- Multi-Car-Support mit separatem Tracking pro Fahrzeug
- Adaptive Geschwindigkeitsprofile basierend auf Crash-Historie
- Anti-Reflection-Mask für bessere Erkennung bei Reflexionen
- Implementierung von Fallback-Mechanismen bei Kameraausfall
- **Multi-Agent-Optimierung**: Aktuell wird Multi-Agent-Training durch mehrere parallele Godot-Instanzen realisiert. Eine Integration direkt in einer Godot-Instanz ohne Headless-Modus würde die Performance verbessern und das Ressourcenmanagement vereinfachen.

10.3.3 Langfristig (Forschung)

- **Domain Randomization**: Diese Technik wurde im Projekt nicht getestet. Durch zufällige Variation von Simulationsparametern (Reibung,

Beleuchtung, Sensorrauscen) während des Trainings könnte die Robustheit des Modells gegenüber realen Bedingungen verbessert werden.

- **Alternative RL-Algorithmen**: Algorithmen wie SAC (Soft Actor-Critic) oder TD3 (Twin Delayed DDPG) könnten bei kontinuierlichen Aktionsräumen Vorteile bieten. Diese wurden im Projektverlauf diskutiert, aber aufgrund von Zeitrestriktionen nicht implementiert.
- **Rohbild-Input mit CNN**: Die Verwendung von Rohbildern als Input für ein Convolutional Neural Network wurde erwogen, jedoch verworfen. Die bereits existierenden Probleme mit Lichtverhältnissen und Tracking hätten sich bei einem bildbasierten Ansatz verstärkt. Zusätzlich wäre die Perspektive und der Abstand der Kamera kritisch, was mit dem aktuellen Ansatz kein Problem darstellt.

11 Begriffserklärungen

Begriff	Definition
Reinforcement Learning	Ein Teilgebiet des maschinellen Lernens, bei dem ein Agent durch fortlaufende Interaktion mit seiner Umgebung und basierend auf Feedback in Form von Belohnungen oder Bestrafungen selbständig erlernt, eine Strategie zur langfristigen Maximierung seines Erfolgs zu entwickeln.
HSV-Farbraum	Ein Farbmodell, das Farben durch Farbton (Hue), Sättigung (Saturation) und Helligkeit (Value) beschreibt. Im Vergleich zum RGB-Farbraum ist HSV robuster gegenüber Beleuchtungsschwankungen, da der Farbton unabhängig von der Helligkeit definiert ist.
AprilTag	Ein zweidimensionales Barcode-System, das speziell für robotische Anwendungen entwickelt wurde. AprilTags ermöglichen die zuverlässige Erkennung und Lokalisierung von Markern in Kamerabildern und werden häufig für Positionsbestimmung und Tracking eingesetzt.
TCP	Transmission Control Protocol – ein verbindungsorientiertes Netzwerkprotokoll, das eine zuverlässige, geordnete Datenübertragung zwischen Computern gewährleistet. Im Projekt wird TCP für die Kommunikation zwischen Python und dem Godot-Simulator verwendet.

Begriff	Definition
IP	Internet Protocol – das grundlegende Protokoll für die Adressierung und das Routing von Datenpaketen in Netzwerken. Eine IP-Adresse (z.B. 127.0.0.1) identifiziert eindeutig einen Netzwerkteilnehmer.
PWM	Pulsweitenmodulation – eine Technik zur Steuerung der durchschnittlichen Leistung eines Signals durch Variation des Tastverhältnisses. Im Projekt wird PWM verwendet, um die Geschwindigkeit des Carrera-Fahrzeugs analog zu steuern.
PPO	Proximal Policy Optimization – ein policy-basierter Reinforcement-Learning-Algorithmus, der für seine Stabilität und gute Performance bei kontinuierlichen Steuerungsaufgaben bekannt ist.
Side-Drag	Die seitliche Kraft, die auf ein Fahrzeug in einer Kurve wirkt. Im Simulator wird der Side-Drag berechnet, um zu erkennen, wann das Fahrzeug die Haftung verliert und von der Strecke abkommt.

12 Aufbauanleitung

Diese Anleitung beschreibt die notwendigen Schritte zur Installation und Inbetriebnahme des Systems.

12.1 Voraussetzungen

12.1.1 Software

- Python 3.10+ mit pip
- Godot 4.5 Engine
- Arduino IDE (für Firmware-Upload)
- Git

12.1.2 Hardware

- Arduino Uno + USB-Kabel
- Logitech Brio 4K Webcam (oder vergleichbar)
- Carrera GO Rennbahn mit AprilTag-markierten Streckenteilen
- Fahrzeug mit grüner Dach-Markierung
- Softbox oder gleichmäßige Beleuchtung

12.2 Installation

12.2.1 1. Repository klonen

```
git clone <repository-url>
cd DS-Carrera-v1
```

12.2.2 2. Python-Abhängigkeiten installieren

```
pip install -r requirements.txt
```

12.2.3 3. Arduino-Firmware flashen

- Arduino IDE öffnen
- Datei `arduino_geschwindigkeit_skaliert/scaled_pwm_controller.ino` laden
- Board auf „Arduino Uno“ setzen
- Korrekten Port auswählen
- Upload durchführen

12.2.4 4. Hardware aufbauen

- Arduino mit Breadboard verbinden
- Pin 11 → PWM-Eingang der Carrera-Bahn
- GND → Gemeinsame Masse
- Status-LEDs (optional): Pin 2 (rot), Pin 4 (blau)

12.2.5 5. Kamera positionieren

- Webcam von oben über der Strecke montieren
- Gesamte Strecke muss im Bild sein
- Kamera-Index in `config.py` anpassen (`CAM_INDEX`)

12.2.6 6. Farbkalibrierung (optional)

Bei abweichenden Lichtverhältnissen die HSV-Werte in `config.py` anpassen:

```
GREEN_LOW = np.array([65, 50, 80])
GREEN_HIGH = np.array([90, 180, 180])
```

12.3 Schnellstart

12.3.1 Nur Simulation (Training)

```
cd CarreraSimulator
python stable_baselines3_example.py --timesteps=100000 --
speedup=32 --n_parallel=8
```

12.3.2 Vollständiges System

```
python launcher/launcher.py --profile=full
```

12.3.3 Inferenz mit trainiertem Modell

```
python stable_baselines3_example.py --inference --
resume_model_path=model.zip
```

12.4 Fehlerbehebung

12.4.1 Kamera wird nicht erkannt

- `CAM_INDEX` in `config.py` ändern (typisch: 0 oder 1)
- Andere Anwendungen schließen, die auf die Kamera zugreifen

12.4.2 AprilTags werden nicht erkannt

- Beleuchtung verbessern (Softbox)

- Kamera-Position und -Fokus prüfen
- Belichtungszeit in `camera_manager.py` anpassen

12.4.3 Arduino antwortet nicht

- Korrekten COM-Port prüfen
- Baudrate muss 9600 sein
- Firmware neu flashen

13 Literatur

[1] E. Beeching, J. Debangoye, O. Simonin, und C. Wolf, „Godot Reinforcement Learning Agents“, 7. Dezember 2021, arXiv: arXiv:2112.03636. doi: 10.48550/arXiv.2112.03636.

[2] „Part 1: Key Concepts in RL — Spinning Up documentation“. Zugegriffen: 25. November 2025. [Online]. Verfügbar unter: https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

[3] R. S. Sutton und A. Barto, Reinforcement learning: an introduction, Second edition. in Adaptive computation and machine learning. Cambridge, Massachusetts London, England: The MIT Press, 2020.

[4] T. C. Kietzmann und M. Riedmiller, „The Neuro Slot Car Racer: Reinforcement Learning in a Real World Setting“, in 2009 International Conference on Machine Learning and Applications, Dez. 2009, S. 311–316. doi: 10.1109/ICMLA.2009.15.